

Trabajo práctico teórico

Nicole Brunstein, Maximo Colombatto, Federico Dimentstein, Ariana Castro y

Agustina Marques Serra.

Universidad tecnológica nacional - UTN BA

Sintaxis y semántica de los lenguajes - K2002

Ing. Roxana Leituz

Fecha: 04/08/2026

Historia de Haskell

¿Qué son los lenguajes funcionales?

Los lenguajes funcionales son lenguajes que basan gran parte de su funcionamiento en el uso de funciones. Una de sus ideas principales es trabajar con expresiones que reciben ciertos datos y generan un resultado, tratando de evitar modificar constantemente los datos originales.

También tienen características como las funciones de orden superior, donde una función puede recibir otra función o devolver una como resultado, y el pattern matching, que permite analizar un dato dependiendo de la forma o estructura que tenga.

En el caso de Haskell también se usa la evaluación perezosa, que básicamente plantea que una expresión no se calcula hasta que realmente se necesita su resultado.

¿Cómo surge Haskell?

Haskell nace por la necesidad de crear un lenguaje común que unificara de alguna forma las principales ideas de los distintos lenguajes funcionales que existían en ese momento.

Durante la década de 1980 hubo un gran crecimiento en la creación de estos lenguajes, pero cada uno tenía distintas características y formas de trabajar, lo que hacía que no hubiera un lenguaje común que sirviera como referencia.

Por esto, en 1987, durante la conferencia Functional Programming Languages and Computer Architecture realizada en Portland, Estados Unidos, un grupo de investigadores planteó la necesidad de crear un nuevo lenguaje funcional que fuera abierto, estándar y común.

La idea no era agarrar todos los lenguajes funcionales y fusionarlos literalmente en uno solo, sino tomar las mejores ideas y características que ya existían para crear un nuevo lenguaje que pudiera servir como base para investigaciones, enseñanza y también para el desarrollo de aplicaciones.

¿Quiénes crearon Haskell y por qué recibió ese nombre?

Para la creación de Haskell se formó un comité internacional integrado por varios investigadores importantes dentro de la programación funcional, entre ellos: Simon Peyton Jones, Paul Hudak, Philip Wadler y John Hughes.

Decidieron nombrar a este lenguaje Haskell en homenaje al matemático y lógico Haskell Brooks Curry, cuyo trabajo sobre lógica matemática y cálculo combinatorio tuvo una gran influencia sobre las bases teóricas de la programación funcional.

¿Cómo evolucionó Haskell?

La primera versión de Haskell apareció en 1990 como Haskell 1.0. Esta versión planteó varias de las bases de lo que conocemos hoy en día como Haskell, como la programación puramente funcional, la evaluación perezosa, un sistema de tipos estático y fuerte, el pattern matching, los tipos de datos algebraicos y la transparencia referencial.

Durante los siguientes años fueron apareciendo nuevas versiones. En 1998 se publicó Haskell 98, una versión más estable y estandarizada, pensada para facilitar la enseñanza del lenguaje y también el desarrollo de compiladores compatibles.

Durante esta etapa también tomó mucha importancia GHC, un compilador de Haskell desarrollado originalmente en la Universidad de Glasgow, que con el tiempo se convirtió en uno de los principales compiladores del lenguaje.

Posteriormente apareció el estándar Haskell 2010, que actualizó Haskell 98 incorporando algunas características que ya eran bastante utilizadas por los programadores.

Desde entonces, la evolución del lenguaje se dio principalmente mediante nuevas extensiones implementadas en GHC.

¿Qué importancia tuvo Haskell?

Como conclusión, Haskell terminó teniendo una influencia muy importante en muchos de los lenguajes de programación que se usan actualmente.

Varias características que en su momento eran más comunes dentro de los lenguajes funcionales, como las funciones lambda, la inmutabilidad o la inferencia de tipos, con el tiempo fueron apareciendo también en lenguajes como JavaScript, Python, Java, Kotlin y C#.

También ideas como el pattern matching y los tipos algebraicos se pueden encontrar hoy en lenguajes más modernos como Rust y Swift.

Otro punto importante de Haskell fue la forma en la que logró trabajar con efectos del mundo real, como la entrada y salida de datos, sin dejar de mantener su idea de lenguaje puramente funcional. Para esto utiliza las mónadas, un concepto que después terminó influyendo en distintas formas de resolver problemas en otros lenguajes.

En conclusión, Haskell comenzó como una forma de crear un lenguaje funcional común para el ámbito académico, pero con el tiempo muchas de sus ideas terminaron influyendo en la forma en la que se diseñaron y evolucionaron otros lenguajes de programación.

Algoritmos

Bubble Sort

El funcionamiento de este algoritmo se basa en ir comparando los elementos de la lista de a pares. Primero se compara el primer elemento con el segundo y, si el primero es mayor que el segundo, intercambian de lugar. Después se compara el segundo con el tercero y se repite el mismo proceso hasta llegar al final de la lista.

De esta forma, en cada recorrido los elementos más grandes van quedando cada vez más cerca del final. Este proceso se repite varias veces hasta que la lista queda completamente ordenada. Si en un recorrido completo no se realiza ningún intercambio, significa que la lista ya está ordenada y el algoritmo puede terminar.

Quick Sort

El funcionamiento de Quick Sort se basa en elegir un elemento de la lista como pivote y, a partir de este, separar los demás elementos en dos grupos. Por un lado quedan los elementos menores al pivote y por el otro los mayores.

Después se vuelve a repetir el mismo proceso con cada uno de esos grupos, eligiendo nuevamente un pivote y separando los elementos.

Esto se repite hasta que los grupos quedan reducidos y finalmente se unen todas las partes, obteniendo la lista completamente ordenada.

Implementación en Haskell y F#

Bubble Sort en Haskell

En Haskell, bubble sort utiliza dos funciones. La función auxiliar bubble se encarga de recorrer la lista de principio a fin comparando los elementos de a dos, si encuentra que el primer elemento es mayor que el segundo, los intercambia de posición. Por su parte, la función principal bubbleSort se encarga de repetir esto una y otra vez sobre la lista resultante. El ciclo se detiene automáticamente en el momento en que una pasada completa por la lista no produce ningún intercambio de lugar, lo que significa que todos los elementos ya han quedado ordenados de menor a mayor.

Bubble Sort en F#

Implementa el ordenamiento de burbuja directamente sobre un arreglo de números (modificando la lista original en lugar de crear una nueva). La función principal tiene;

- Swap: permite intercambiar dos elementos en un array.
- Bucle de ordenamiento: es el ciclo principal que recorre repetidamente el arreglo comparando parejas de números vecinos de izquierda a derecha. Si encuentra que un número es mayor que el que tiene al lado, los intercambia de lugar usando [swap] y anota que hubo un cambio.

Quick Sort en Haskell

En Haskell, la manera de aplicar quicksort es muy elegante, pero tiene ciertas desventajas con respecto a muchos otros lenguajes que no se limitan al paradigma funcional. En Haskell, se utilizan listas enlazadas inmutables y tiene la particularidad de analizar las cosas cuando las necesita o se las llama (Lazy Evaluation). Básicamente se utiliza la recursión para dividir la lista en dos partes, y luego cada lista de esas dos se vuelve a subdividir en dos partes, y así sucesivamente, para luego concatenar todo.

Quick Sort en F#

Por otro lado, F# posee datos mutables ya que utiliza arrays y no evalúa de forma tardía cuando es necesario sino que analiza las expresiones apenas son llamadas o solicitadas. Funciona muy similar al modelo elegante de Haskell:

- Swap: permite intercambiar dos elementos en un array.
- Sort: ordena la lista en dos mitades a partir del último elemento (pivote) separando los menores y los mayores. Y recursivamente ordena las mitades por separado hasta tener la lista ordenada.

Datos

Quick Sort

```
$ ./quicksort_hs.exe
Haskell Quicksort Performance Test
Generating 100000 random elements...
Sorting...
Sorted successfully! Time elapsed: 0.115578 seconds
```

0,11 seg (quicksort en haskell)

```
$ dotnet run -c Release
F# Quicksort (In-place) Performance Test
Generating 100000 random elements...
Sorting...
Sorted successfully! Time elapsed: 0.015147 seconds
```

0,01 seg (quicksort en F#)

Elementos	Haskell	F#
100.000	0,11 s	0,01 s

Bubble Sort

```
$ ./bubblesort_hs.exe
Haskell Bubble Sort Performance Test
Generating 10000 random elements...
Sorting...
Sorted successfully! Time elapsed: 1.420433 seconds
```

1,42 seg (bubble sort en haskell)

```
$ dotnet run -c Release
F# Bubble Sort (In-place) Performance Test
Generating 10000 random elements...
Sorting...
Sorted successfully! Time elapsed: 0.113954 seconds
```

0,11 seg (bubble sort en F#)

Elementos	Haskell	F#
10.000	1,42 s	0,11 s

Análisis de los datos calculados

Para obtener los resultados utilizamos un programa que crea una lista aleatoria de elementos, y ejecutamos los archivos de cada lenguaje de manera compilada y optimizada lo más posible para que la única diferencia de rendimiento sea en lo posible por el lenguaje en sí.

En el caso de Bubble Sort (evaluado con 10.000 elementos aleatorios/desordenados), F# obtuvo un desempeño sustancialmente mejor que Haskell. Mientras que Haskell tardó 1,420433 segundos, F# completó la ordenación en 0,113954 segundos (12,5 veces más rápido).

En el caso de Quick Sort (evaluado con 100.000 elementos aleatorios/desordenados), F# también obtuvo tiempos sensiblemente menores. Haskell registró un tiempo de 0,115578 segundos, mientras que en F# tardó 0,015147 segundos (7,6 veces más rápido).

En ambos algoritmos, F# mostró una clara ventaja en tiempo de ejecución.

Posibles causas de las diferencias de rendimiento entre Haskell y F#

En las pruebas realizadas, F# obtuvo menores tiempos que Haskell en los dos algoritmos. Aunque en ambos casos se aplicaron los mismos métodos de ordenamiento, cada lenguaje los implementó utilizando estructuras y mecanismos diferentes.

Una de las diferencias se encuentra en el modelo de evaluación. F# utiliza normalmente evaluación estricta, por lo que las expresiones se calculan a medida que se ejecuta el programa. Haskell utiliza evaluación perezosa, lo que significa que ciertos cálculos pueden quedar pendientes hasta que su resultado sea necesario. Estas expresiones pendientes, conocidas como *thunks*, pueden generar un costo adicional si se acumulan durante la ejecución.

Otra diferencia importante es la estructura de datos utilizada. En Haskell se trabajó con listas enlazadas inmutables. Como sus elementos no pueden modificarse directamente, los algoritmos deben construir nuevas listas durante las llamadas recursivas.

En F# se utilizaron arrays mutables, que permiten acceder a los elementos por posición e intercambiarlos dentro de la misma estructura. Esto reduce la creación de colecciones intermedias y favorece el rendimiento en algoritmos de ordenamiento basados en intercambios.

En Quick Sort también existen diferencias en la forma de particionar los datos. La implementación de Haskell utiliza filtrado y concatenación de listas, lo que implica realizar varios recorridos y crear nuevas estructuras. En F#, la partición se realiza directamente sobre el array mediante intercambios, evitando parte de ese trabajo adicional.

Por último, también influyen los compiladores, el manejo de memoria y las optimizaciones aplicadas por GHC y por .NET.

Por estas razones, F# obtuvo mejores tiempos bajo las condiciones utilizadas. Sin embargo, los resultados no permiten afirmar que F# sea siempre más rápido que Haskell, sino que la implementación elegida en F# fue más eficiente para estas pruebas.

Conclusión

A partir de las pruebas realizadas pudimos comparar el rendimiento de Haskell y F# mediante implementaciones de Bubble Sort y Quick Sort adaptadas a las características de cada lenguaje.

En Bubble Sort, F# completó el ordenamiento de 10.000 elementos en 0,113954 segundos, mientras que Haskell tardó 1,420433 segundos. En Quick Sort, F# ordenó 100.000 elementos en 0,015147 segundos y Haskell lo hizo en 0,115578 segundos.

En ambos casos, F# obtuvo menores tiempos. Esta diferencia puede estar relacionada con el uso de arrays mutables y operaciones in-place, que permiten modificar los elementos sin construir nuevas colecciones. En Haskell, el uso de listas inmutables, filtrado, concatenación y evaluación perezosa puede agregar creación de estructuras y trabajo adicional.

También influyeron los compiladores y las optimizaciones utilizadas. Haskell fue compilado mediante GHC con la opción -O2, mientras que F# se ejecutó mediante .NET en modo Release.

En conclusión, bajo las implementaciones y condiciones elegidas, F# mostró un mejor rendimiento que Haskell en los dos algoritmos evaluados. Sin embargo, no se puede afirmar que F# sea siempre más rápido, ya que los resultados también dependen de las estructuras de datos, de la implementación elegida y del entorno de ejecución.

Bibliografía

Documentación y especificaciones oficiales

Peyton Jones, Simon (ed.) y otros. *Haskell 98 Language and Libraries: The Revised Report – Introduction.*

<https://www.haskell.org/onlinereport/intro.html>

Peyton Jones, Simon (ed.) y otros. *Haskell 98 Language and Libraries: The Revised Report – Syntax Reference.*

<https://www.haskell.org/onlinereport/syntax-iso.html>

Haskell.org. *Haskell 2010 Language Report – Chapter 10: Syntax Reference.*

<https://www.haskell.org/onlinereport/haskell2010/haskellch10.html>

F# Software Foundation. *The F# Language Specification.*

<https://fsharp.org/specs/language-spec/>

F# Language Specification. *Expressions.*

<https://fsharp.github.io/fslang-spec/expressions/>

F# Language Specification. *Patterns.*

<https://fsharp.github.io/fslang-spec/patterns/>

Historia y características de Haskell

Serokell. *Haskell: History of a Community-Powered Language.*

<https://serokell.io/blog/haskell-history>

Wikipedia. *Haskell.*

<https://es.wikipedia.org/wiki/Haskell>

Recursos audiovisuales

un simple Dev. ✨ ALGORITMO de ordenamiento ● BUBBLE SORT o BURBUJA ●

<https://www.youtube.com/watch?v=HD6DdJKpXnA>

Chio Code. Quick Sort | Ordenamiento Rápido | Explicado con Cartitas!

<https://www.youtube.com/watch?v=UrPJLhKF1jY>

Programando Otra Historia. Hablemos de HASKELL

https://www.youtube.com/watch?v=_xvbAPdhciU